

An Experimental Study of Parallel Sorting by Regular Sampling on Shared-Memory Systems

Saumya Patel

Student Number: 1812526

CCID: snp2

1 Introduction

This report describes the implementation and experimental evaluation of Parallel Sorting by Regular Sampling (PSRS) in a shared-memory environment using POSIX threads. The implementation was evaluated on randomly generated integer arrays ranging from one million to sixty-four million keys, with worker threads varied from 2 to 256. All experiments were compiled with aggressive optimization flags (`-O3 -march=native`) to ensure that the baseline performance reflects realistic production code. Rather than focusing solely on theoretical asymptotic bounds, this work examines how the algorithm behaves in practice on a modern multicore system, specifically the Apple M4 architecture.

In addition to measuring overall execution time and speedup, the experiments record the execution time of each phase of the algorithm. This phase-level instrumentation makes it possible to observe how individual components scale and how their relative costs change as the level of parallelism increases. The results demonstrate that PSRS achieves a peak speedup of $4.67\times$ at 32 threads, after which scalability is limited by hardware constraints. Phase-level measurements reveal that while the local sorting phase scales linearly, the later stages involving synchronization and data movement increasingly dominate execution time, creating a "memory wall" that prevents linear scaling at high thread counts.

2 Implementation Details

This section describes the practical design choices made while implementing PSRS. The focus is on how the abstract algorithm was realized in code to maximize efficiency on shared-memory hardware.

2.1 Thread Model and Phase Structure

The implementation follows a single-program multiple-data (SPMD) execution model, where a fixed number of worker threads equal to the requested processor count execute the same code on disjoint portions of the input array. Threads are created once at the start of execution and persist across all phases of the algorithm.

Phase boundaries are enforced using barrier synchronization, ensuring that all threads complete a phase before proceeding to the next one. This monolithic thread pool approach avoids the overhead of repeated thread creation and destruction (fork/join), which can significantly impact performance at higher thread counts where the work-per-thread is small.

2.2 Synchronization and Phase Boundaries

Synchronization is strictly limited to phase boundaries. Within each phase, threads operate independently on private or disjoint memory regions without fine-grained locking. Barriers are used to ensure that all threads complete a phase before shared data structures, such as the array of pivots or the count of elements per bucket, are accessed by subsequent phases.

Although barrier synchronization introduces overhead (latency increases as $O(\log p)$ or $O(p)$ depending on the barrier implementation), limiting synchronization to a small number of well-defined points helps contain its impact. However, at high thread counts ($p > 64$), the cost of threads "checking in" to the barrier begins to constitute a measurable fraction of the runtime.

2.3 Sampling and Pivot Selection

During the local sorting phase, each thread selects regular samples from its locally sorted subarray. These samples are collected into a shared array and sorted by the primary thread to determine global pivot values.

A critical detail often overlooked is that pivot selection is not perfectly uniform. Small imbalances in bucket sizes can arise due to the randomness of the input data and the finite number of samples (p^2). While these imbalances are negligible at low concurrency, they become more visible at higher thread counts ($p = 256$), where even a small variance in bucket size can cause significant "straggler" effects, forcing the entire system to wait for the most overloaded thread during the merge phase.

2.4 Data Movement and k -Way Merge Strategy

After partitioning, each thread contributes portions of its local data to multiple buckets. These buckets are later merged to form the final sorted output. This phase involves substantial data movement and non-contiguous memory access, making it the most sensitive to memory bandwidth limitations.

To efficiently combine the partially ordered sublists, a **heap-based k -way merge strategy** is used. Unlike a naive iterative approach (which merges list 1 and 2, then the result with list 3, etc.), the k -way merge maintains a min-heap of the head elements from all p incoming partitions.

- **Efficiency:** The k -way merge requires only a single pass over the data to produce the final sorted sequence. In contrast, iterative 2-way merges require $\log p$ passes, dramatically increasing memory traffic.
- **Complexity:** Each element is touched exactly once read and once written, with $O(\log p)$ comparisons per element.

Given that the memory bus is the primary bottleneck on modern multicore systems, minimizing the number of read/write passes is far more important than minimizing CPU instructions.

Phase-level instrumentation (plots omitted in this version) shows that while local sorting scales effectively with p , the later stages involving data movement and synchronization grow disproportionately and become the dominant bottleneck at higher thread counts.

2.5 Instrumentation and Measurement

Execution time is measured using `gettimeofday` to capture real wall-clock elapsed time. To ensure statistical significance and account for "cache warming" effects, each experimental configuration is executed 7 times. The first 2 runs are discarded to allow the branch predictor and cache hierarchy to stabilize, and the average of the remaining 5 runs is reported. Correctness is verified by comparing the parallel output against a standard `qsort` reference implementation, ensuring that performance measurements are not obtained at the expense of accuracy.

3 Experimental Results and Analysis

This section presents and analyzes the performance results. All experiments explicitly distinguish between **software threads** and **physical CPU cores**; thread counts were intentionally varied beyond the number of available cores to evaluate oversubscription effects on shared-memory performance.

3.1 Speedup as a Function of Thread Count

For small thread counts ($2 \leq p \leq 16$), PSRS scales effectively. Increasing the number of threads yields a steady improvement, indicating that the computationally intensive local sorting phase benefits strongly from parallelism.

However, beyond 32 threads, the marginal gains diminish substantially. While speedup peaks at 32 threads ($4.67\times$), it plateaus and eventually declines at 256 threads. This behavior indicates that overheads associated with synchronization, thread scheduling, and shared memory contention increasingly dominate execution time as parallelism grows.

3.2 Deep Dive: The Cost of Oversubscription

Performance degradation beyond 64 threads stems from severe hardware oversubscription. On the 12-core Apple M4, running 64 threads forces > 5 software threads to contend for each physical core. This incurs three major penalties: **context switching** overhead wastes CPU cycles; **cache pollution** degrades performance as rotating threads evict their predecessors' L1/L2 cache lines; and **barrier stragglers** arise when a descheduled thread stalls the entire global synchronization. Consequently, adding threads beyond the physical limit amplifies coordination costs without increasing throughput.

3.3 Sequential Versus Best Parallel Performance

Across input sizes, scalability improves with larger workloads because the $O(N \log N)$ local sorting time dominates the $O(P)$ synchronization costs. Small inputs (1M) show limited speedup (about $2.59\times$), while larger inputs (32M) achieve higher speedup (about $4.67\times$).

The optimal thread count varies with input size (often between 16, 32, and 128), likely due to cache effects and the balance between useful work and overhead.

3.4 Runtime as a Function of Thread Count

Runtime drops sharply when increasing threads from 2 to 16. However, between 32 and 128 threads, runtime decreases only marginally. This suggests that the system has reached the "memory wall", the point where the CPU cores can process data faster than the main memory can supply it. Adding more threads at this stage simply increases contention for the memory bus without increasing throughput.

3.5 Runtime as a Function of Input Size

Runtime increases with input size for all thread counts, and the benefit of parallelism is most visible at larger N . For small workloads ($N < 4M$), very high thread counts (128, 256) can perform worse than moderate thread counts because overhead dominates when work-per-thread becomes too small.

3.6 Efficiency and Amdahl's Law

The observed performance limits are well explained by Amdahl's Law, which states that the maximum speedup $S(p)$ is bounded by the serial fraction f of the program:

$$S(p) = \frac{1}{(1 - f) + \frac{f}{p}}$$

Using experimental data for $N = 32M$, the sequential time is $2.79s$ and the time at $p = 32$ is $0.59s$. Solving for f suggests an effective parallel fraction of approximately 90%.

If the code were perfectly parallelizable ($f = 1.0$), we would expect a speedup of $32\times$. The fact that we only achieve $4.67\times$ indicates that the "serial" portion, comprising barrier synchronization, pivot selection, and memory bus contention, is significant. Specifically, as p increases, the memory bandwidth per core decreases. This means that f is not actually a constant; the "cost" of memory operations increases with p , making the effective serial portion larger at high thread counts. This explains why the speedup curve not only flattens but eventually dips (efficiency plot omitted in this version).